

Reviewer Pack — Polymatroid Rank Lower Bound for Monotone CLIQUE

Entry 834e913154a8 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-10 07:36:46 UTC

Polymatroid Rank Lower Bound for Monotone CLIQUE

Entry ID: 834e913154a8 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-10 07:36:46 UTC

1. Conjecture

Field A (mathematical branch): Polymatroid Theory **Field B** (complexity object): Monotone Circuit Size for k-CLIQUE

Statement:

For any graph G on n vertices, let $M(G)$ be the polymatroid defined by the rank function $r(S) = \max\{|S \cap C| : C \text{ is a clique in } G\}$. Then $r(M(G)) \geq \Omega(n^{\{1/2\}})$ for all graphs G with clique number $\leq n^{\{1/2\}} + 1$.

Rationale (proposer's reasoning):

Polymatroid rank captures submodular constraints inherent in clique enumeration. By linking this geometric invariant to monotone circuit complexity, we expose structural limitations in representing clique problems with AND/OR gates, potentially revealing new barriers for monotone lower bounds.

Taxonomy category: GCT (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 2638711d55bdb73a

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.95	SAFE	UNCERTAIN
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.95	UNCERTAIN	SAFE
KARP_LIPTON	SAFE	0.95	SAFE	UNCERTAIN

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
---------	---------	------------	------------------	------------------

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=124, elapsed=240.1s

5.1 Generated Python source

```
import random
import math

def generate_random_graph(n):
    edges = set()
    for i in range(n):
        for j in range(i + 1, n):
            if random.random() < 0.5:
                edges.add((i, j))
    return edges

def is_clique(graph, vertices):
    for u, v in itertools.combinations(vertices, 2):
        if (u, v) not in graph and (v, u) not in graph:
            return False
    return True

def find_max_clique_size(graph, n):
    max_clique_size = 0
    for i in range(1, n + 1):
        for vertices in itertools.combinations(range(n), i):
            if is_clique(graph, vertices):
                max_clique_size = max(max_clique_size, len(vertices))
    return max_clique_size

def greedy_polymatroid_rank(graph, n):
    rank = 0
    covered_edges = set()
    for _ in range(n):
        max_gain = -1
        best_vertex = None
        for v in range(n):
            if v not in covered_edges:
                gain = sum(1 for u in range(n) if (u, v) in graph and u not in covered_edges)
                if gain > max_gain:
                    max_gain = gain
                    best_vertex = v
        rank += max_gain
        covered_edges.update((best_vertex, u) for u in range(n) if (best_vertex, u) in graph or (u, best_vertex) in graph)
    return rank
```

```

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = 40
    clique_number = math.isqrt(n) + 1
    graph = generate_random_graph(n)
    while find_max_clique_size(graph, n) > clique_number:
        graph = generate_random_graph(n)

    rank = greedy_polymatroid_rank(graph, n)
    return {
        "metric_name": "polymatroid_rank",
        "metric_value": rank,
        "instances_tested": 1,
        "conjecture_holds": rank >= 6.4,
        "counterexample": "" if rank >= 6.4 else f"Graph with clique number {clique_number} and ra
    }

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] or [2**i - 1 for i in range(5, 35)]
    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {{\"seed\": {seed}, \"metric_name\": \"{result['metric_name']}\", \"metric_
        results.append(result)

    mean_rank = sum(r["metric_value"] for r in results) / len(results)
    std_rank = math.sqrt(sum((r["metric_value"] - mean_rank) ** 2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_rank} std={std_rank} support_fraction={support_fractions}")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_rank} std={std_rank} support_fraction={support_fractions}")
    else:
        first_failing_seed = next(seed for seed, r in zip(seeds, results) if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"{results[seeds.index(first_failing_seed)][\"counterexample\"]}\"")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test timed out without producing results, preventing evaluation of support fraction or counterexamples. | next: Run test with extended timeout and higher n values to gather conclusive data

11. Audit log (LLM calls)

Total LLM calls: 8

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	111407
2	propose	ollama_remote	qwen3:8b	0	0	54818
3	preregistration	ollama_remote	qwen3:8b	0	0	24134
4	novelty	ollama_remote	qwen3:8b	0	0	20700
5	novelty	ollama_remote	qwen3:8b	0	0	16999
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10810
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9746
8	judge	ollama_remote	qwen3:8b	0	0	17654

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 266268 ms total latency. Provider mix: {'ollama_remote': 8}

(full prompt+response transcripts available in research/audit/834e913154a8.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/834e913154a8.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/834e913154a8.tar.gz (if generated)